

Формы в React

Введение в работу с формами

Формы являются важной частью многих веб-приложений. В React работа с формами немного отличается от традиционного HTML, поскольку состояние формы обычно хранится в компонентах React.

Контролируемые компоненты

В HTML элементы формы, такие как `<input>`, `<textarea>` и `<select>`, обычно поддерживают собственное состояние и обновляют его на основе пользовательского ввода. В React мутабельное состояние обычно хранится в свойстве `state` компонентов и обновляется только с помощью `setState()` или хука `useState`.

Контролируемый компонент — это компонент, в котором значения элементов формы контролируются React.

```
import React, { useState } from 'react';

function NameForm() {
  const [value, setValue] = useState('');

  const handleChange = (event) => {
    setValue(event.target.value);
  };

  const handleSubmit = (event) => {
    alert('Отправленное имя: ' + value);
    event.preventDefault();
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Имя:
        <input type="text" value={value} onChange={handleChange} />
      </label>
      <input type="submit" value="Отправить" />
    </form>
  );
}
```

Различные типы элементов формы

Textarea

В HTML элемент `<textarea>` определяет свой текст дочерним элементом. В React `<textarea>` использует атрибут `value`.

```
function EssayForm() {
  const [essay, setEssay] = useState('Напишите эссе о вашем любимом DOM-элементе. ');

  const handleChange = (event) => {
    setEssay(event.target.value);
  };

  return (
    <form>
      <label>
        Эссе:
        <textarea value={essay} onChange={handleChange} />
      </label>
    </form>
  );
}
```

Select

В HTML `<select>` создаёт выпадающий список. В React вместо использования атрибута `selected` у тега `<option>`, используется атрибут `value` у корневого тега `<select>`.

```
function FlavorForm() {
  const [flavor, setFlavor] = useState('coconut');

  const handleChange = (event) => {
    setFlavor(event.target.value);
  };

  return (
    <form>
      <label>
        Выберите ваш любимый вкус:
        <select value={flavor} onChange={handleChange}>
          <option value="grapefruit">Грейпфрут</option>
          <option value="lime">Лайм</option>
          <option value="coconut">Кокос</option>
          <option value="mango">Манго</option>
        </select>
      </label>
    </form>
  );
}
```

Множественный выбор

В `<select>` можно также указать атрибут `multiple`, что позволит пользователю выбрать несколько опций.

```
<select multiple={true} value={['B', 'C']}>
```

Обработка нескольких элементов ввода

Когда вам нужно обрабатывать несколько контролируемых элементов `input`, вы можете добавить атрибут `name` каждому элементу и позволить функции-обработчику выбрать, что делать, основываясь на значении `event.target.name`.

```
function Reservation() {
  const [state, setState] = useState({
    isGoing: true,
    numberOfGuests: 2
  });

  const handleInputChange = (event) => {
    const target = event.target;
    const value = target.type === 'checkbox' ? target.checked : target.value;
    const name = target.name;

    setState({
      ...state,
      [name]: value
    });
  };

  return (
    <form>
      <label>
        Пойдут:
        <input
          name="isGoing"
          type="checkbox"
          checked={state.isGoing}
          onChange={handleInputChange} />
      </label>
      <br />
      <label>
        Количество гостей:
        <input
          name="numberOfGuests"
          type="number"
          value={state.numberOfGuests}
          onChange={handleInputChange} />
      </label>
    </form>
  );
}
```

```
    </form>
  );
}
```

Неконтролируемые компоненты

В большинстве случаев рекомендуется использовать контролируемые компоненты. Однако иногда использование неконтролируемых компонентов может быть проще, особенно при интеграции React-кода с кодом, не относящимся к React.

В неконтролируемом компоненте данные формы обрабатываются самим DOM. Вместо того, чтобы писать обработчик событий для каждого обновления состояния, можно использовать `ref` для получения значений формы из DOM.

```
import React, { useRef } from 'react';

function FileInput() {
  const fileInput = useRef(null);

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Выбранный файл - ${fileInput.current.files[0].name}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Загрузить файл:
        <input type="file" ref={fileInput} />
      </label>
      <br />
      <button type="submit">Отправить</button>
    </form>
  );
}
```

Валидация форм

Валидация форм может быть реализована различными способами:

Встроенная валидация HTML5

HTML5 предоставляет встроенные атрибуты валидации, такие как `required`, `pattern`, `min`, `max` и т.д.

```
<input type="email" required pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$" />
```

Пользовательская валидация

Вы можете реализовать собственную логику валидации в React.

```
function ValidationForm() {
  const [email, setEmail] = useState('');
  const [error, setError] = useState(null);

  const validateEmail = (email) => {
    const re = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
    return re.test(String(email).toLowerCase());
  };

  const handleChange = (event) => {
    const value = event.target.value;
    setEmail(value);

    if (value && !validateEmail(value)) {
      setError('Пожалуйста, введите корректный email');
    } else {
      setError(null);
    }
  };

  return (
    <form>
      <label>
        Email:
        <input type="text" value={email} onChange={handleChange} />
      </label>
      {error && <div style={{ color: 'red' }}>{error}</div>}
    </form>
  );
}
```

Библиотеки для работы с формами

Для сложных форм часто используются специализированные библиотеки:

Formik

Formik — одна из самых популярных библиотек для работы с формами в React. Она помогает с:

- Получением значений в/из состояния формы
- Валидацией и сообщениями об ошибках
- Обработкой отправки формы

```
import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';
```

```

const SignupSchema = Yup.object().shape({
  firstName: Yup.string()
    .min(2, 'Слишком короткое имя!')
    .max(50, 'Слишком длинное имя!')
    .required('Обязательное поле'),
  lastName: Yup.string()
    .min(2, 'Слишком короткая фамилия!')
    .max(50, 'Слишком длинная фамилия!')
    .required('Обязательное поле'),
  email: Yup.string()
    .email('Некорректный email')
    .required('Обязательное поле'),
});

function SignupForm() {
  return (
    <Formik>
      initialValues={{ firstName: '', lastName: '', email: '' }}
      validationSchema={SignupSchema}
      onSubmit={values => {
        alert(JSON.stringify(values, null, 2));
      }}
    >
    <{({ errors, touched }) => (
      <Form>
        <div>
          <label htmlFor="firstName">Имя</label>
          <Field name="firstName" />
          <ErrorMessage name="firstName" component="div" className="error" />
        </div>

        <div>
          <label htmlFor="lastName">Фамилия</label>
          <Field name="lastName" />
          <ErrorMessage name="lastName" component="div" className="error" />
        </div>

        <div>
          <label htmlFor="email">Email</label>
          <Field name="email" type="email" />
          <ErrorMessage name="email" component="div" className="error" />
        </div>

        <button type="submit">Отправить</button>
      </Form>
    )}
  </Formik>
  );
}

```

React Hook Form

React Hook Form — это библиотека, которая фокусируется на производительности и минимизации перерисовок при работе с формами.

```
import { useForm } from 'react-hook-form';

function HookForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();
  const onSubmit = data => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <div>
        <label>Имя</label>
        <input {...register('firstName', { required: true, maxLength: 20 })} />
        {errors.firstName?.type === 'required' && <span>Это поле
обязательно</span>}
        {errors.firstName?.type === 'maxLength' && <span>Имя не может быть
длиннее 20 символов</span>}
      </div>

      <div>
        <label>Фамилия</label>
        <input {...register('lastName', { pattern: /^[A-Za-z]+$/i })} />
        {errors.lastName && <span>Фамилия должна содержать только буквы</span>}
      </div>

      <div>
        <label>Возраст</label>
        <input type="number" {...register('age', { min: 18, max: 99 })} />
        {errors.age && <span>Возраст должен быть от 18 до 99</span>}
      </div>

      <input type="submit" />
    </form>
  );
}
```

Заключение

Работа с формами в React предоставляет гибкие возможности для управления пользовательским вводом. Контролируемые компоненты обеспечивают более предсказуемое поведение и позволяют легко реализовать валидацию, но требуют больше кода. Неконтролируемые компоненты проще в использовании, но предоставляют меньше контроля.

Для сложных форм рекомендуется использовать специализированные библиотеки, такие как Formik или React Hook Form, которые значительно упрощают работу с формами, валидацией и обработкой ошибок.

